



Nombre y Apellidos:
Sección:

RUT:
Firma:

CC1002 – 2025 Semestre Primavera - Control 1 - Duración: 2:00 horas

Indicaciones Generales:

- 1.- Lea atentamente las preguntas. **Habrá dos rondas de preguntas de enunciado durante el control.**
- 2.- Debe responder en las hojas de respuestas que se le entregarán – no se aceptarán hojas adicionales.
- 3.- Siempre debe incluir la receta de diseño al escribir una función, implementando al menos un test.

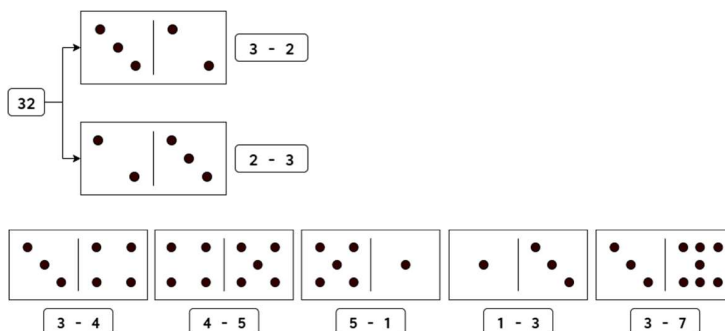
Pregunta 1 (50%)

El **domino** es un clásico juego de mesa que utiliza un conjunto de 28 fichas, cada una dividida en dos espacios con puntos (o "ojos") que van del 0 al 6, cubriendo todas las combinaciones posibles.

Importante: Para este problema el dominó tendrá valores entre 1 y 7.

Los jugadores colocan por turnos las fichas de forma que los puntos de los extremos coincidan con los de las fichas ya puestas en la mesa, siendo el objetivo ser el primero en quedarse sin fichas o tener la menor cantidad de puntos si el juego se "cierra".

- Una **ficha** se representa como un par de dígitos, cada uno entre 1 y 7.
Por ejemplo el número **32** representa una ficha, que podría ser colocada en el juego como **3-2** o **2-3**.
- Un **juego** se representa como un entero positivo que registra los valores de las fichas en el tablero. Por ejemplo, el número **3445511337** representa las siguientes fichas en juego:



Se dispone del módulo **domino**, con las siguientes funciones:

Función	Propósito	Ejemplo
primer_valor(juego)	retorna el valor libre de la primera ficha	primer_valor(3445511337) == 7
ultimo_valor(juego)	retorna el valor libre de la última ficha	ultimo_valor(3445511337) == 3
poner_inicio(ficha, juego)	recibe una ficha y la pone al inicio del juego	poner_inicio(71, 3445511337) == 3445511337 71 poner_inicio(17, 3445511337) == 3445511337 71 poner_inicio(26, 3445511337) == 3445511337
poner_final(ficha, juego)	recibe una ficha y la pone al final del juego	poner_final(23, 3445511337) == 23 3445511337 poner_final(32, 3445511337) == 23 3445511337 poner_final(26, 3445511337) == 3445511337

Para las dos últimas funciones, notar que, si no es posible colocar la ficha, se retorna el mismo juego que se recibió sin modificaciones. Con esto, haga lo siguiente:

(A) (3.0p) Escriba la función **poner_inicio(ficha, juego)** del módulo **domino**.

(B) (3.0p) Usando el módulo **domino**, escriba la función **poner_ficha(ficha, juego)**, donde **ficha** es el valor de dos dígitos que representa la ficha a poner, e intenta colocarla en alguno de los extremos del juego. Si la ficha se puede poner en ambos lados, usted elija uno. Si no se puede poner la ficha, se retorna el mismo juego que se recibió.

Ejemplos:

- poner_ficha(32, 3445511337) entrega: **23**3445511337 (se colocó la ficha 32 como 2-3 al extremo izquierdo del juego)
- poner_ficha(56, 3445511337) entrega: 3445511337 (no se pudo colocar la ficha 5-6 en alguno de sus extremos)

Indicación: Puede usar todas las funciones del módulo **domino** sin necesidad de definir las.

NOTA: no se puede transformar el número entero a String o viceversa para resolver esta pregunta.



Nombre y Apellidos:
Sección:

RUT:
Firma:

Solución Pregunta 1:

(A) Asignación de puntajes:

- **(0.2p)** Contrato
- **(0.1p)** Propósito + Ejemplo
- **(0.1p)** Encabezado función
- **(0.2p)** Testing
- Cuerpo función:
 - **(0.6p)** Obtener valor al inicio
 - **(0.2p)** Probar si se puede jugar con el primer número de la ficha
 - **(0.4p)** Devolver juego con esta opción
 - **(0.2p)** Probar si se puede jugar con el segundo número de la ficha
 - **(0.4p)** Devolver juego con esta opción
 - **(0.6p)** Devolver juego sin cambios

Implementación de ejemplo:

```
# poner_inicio: int int --> int
# Recibe una ficha y la pone al inicio del juego.
# Ejemplo:
# poner_inicio(71, 3445511337) retorna 344551133771
# poner_inicio(17, 3445511337) retorna 344551133771
# poner_inicio(26, 3445511337) retorna 3445511337
def poner_inicio(ficha, juego):
    d1 = ficha % 10
    d2 = ficha // 10
    valor = primer_valor(juego)
    if valor == d1:
        return juego*100 + d1*10 + d2
    elif valor == d2:
        return juego*100 + d2*10 + d1
    else:
        return juego

# Test
assert poner_inicio(71, 3445511337) == 344551133771
assert poner_inicio(17, 3445511337) == 344551133771
assert poner_inicio(26, 3445511337) == 3445511337
```



Nombre y Apellidos:
Sección:

RUT:
Firma:

(B) Asignación de puntajes:

- **(0.2p)** Contrato
- **(0.1p)** Propósito + Ejemplo
- **(0.1p)** Encabezado función
- **(0.2p)** Testing
- **(0.1p)** Importar módulo domino
- Cuerpo función:
 - **(0.3p)** Probar si se puede jugar al inicio con la ficha
 - **(0.5p)** Devolver juego con esta opción
 - **(0.3p)** Probar si se puede jugar al final con la ficha
 - **(0.5p)** Devolver juego con resta opción
 - **(0.7p)** Devolver juego sin cambios

Implementación de ejemplo:

```
from modulo import *

# poner_ficha: int int --> int
# pone la ficha en el juego y retorna el nuevo juego.
# Ejemplo: poner_ficha(17, 3445511337) retorna 344551133771
#           poner_ficha(32, 3445511337) retorna 233445511337
def poner_ficha(ficha, juego):
    d1 = ficha % 10
    d2 = ficha // 10
    valor = primer_valor(juego)
    if valor == d1 or valor == d2:
        return poner_inicio(ficha, juego)
    valor = ultimo_valor(juego)
    if valor == d1 or valor == d2:
        return poner_final(ficha, juego)
    return juego

# Test
assert poner_ficha(17, 3445511337) == 344551133771
assert poner_ficha(32, 3445511337) == 233445511337

# Solucion alternativa 1
def poner_ficha(ficha, juego):
    #intento ponerla la inicio
    juego1 = poner_inicio(ficha, juego)
    if juego1 == juego: #no la puso
        juego1 = poner_final(ficha, juego)
    return juego1

# Solucion alternativa 2
def poner_ficha(ficha, juego):
    return max(poner_inicio(ficha, juego), poner_final(ficha, juego))
```



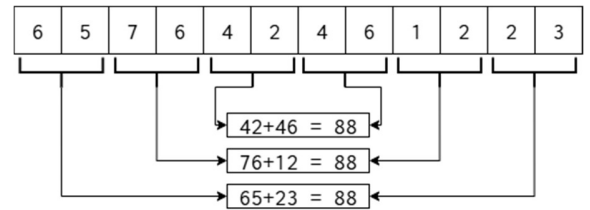
Nombre y Apellidos:
Sección:

RUT:
Firma:

Pregunta 2 (50%)

Los números de Sumeru son los que cumplen la siguiente propiedad:

- Su cantidad de dígitos es múltiplo de 4.
- Si se toma el primer par de dígitos del número, y se suma con el último par, tiene que dar la misma suma que al segundo par de dígitos del número sumarle el penúltimo par de dígitos, y así sucesivamente. Lo cual se puede ver en el diagrama de la derecha.
- Todos los números con exactamente 4 dígitos por definición se consideran números de Sumeru.



Para esta pregunta puede asumir como conocida la función **digitos(N)**, que permite contar la cantidad de dígitos que tiene un número entero positivo.

- `digitos(5421)` entrega 4.
- `digitos(9)` entrega 1.

(A) (3 ptos.) Escriba la función **sumarExtremos(N)**, que recibe un número entero positivo con una cantidad de dígitos mayor o igual a 4, y entrega la suma de los pares de dígitos en sus extremos. Ejemplos:

- `sumarExtremos(75412698)` entrega 173 (que corresponde a sumar 75 + 98).
- `sumarExtremos(12453164)` entrega 76 (que corresponde a sumar 12 + 64).
- `sumarExtremos(5642)` entrega 98 (que corresponde a sumar 56 + 42).
- `sumarExtremos(657642461223)` entrega 88 (que corresponde a sumar 65 + 23).

(B) (3 ptos.) Escriba la función recursiva **comprobar(N)**, que recibe un número entero positivo cuya cantidad de dígitos es múltiplo de 4, y verifica que se cumpla la propiedad de los números de Sumeru. Ejemplos:

- `comprobar(75412698)` entrega False (ya que $75 + 98 \neq 41 + 26$).
- `comprobar(12453164)` entrega True (ya que $12 + 64 == 45 + 31$).
- `comprobar(5642)` entrega True (por definición).
- `comprobar(657642461223)` entrega True (ya que $65 + 23 == 76 + 12 == 42 + 46$).

Indicaciones:

- Note que la función ya recibe un número cuya cantidad de dígitos es múltiplo de 4. No es necesario que lo valide dentro de la función.
- Puede usar la función `sumarExtremos` sin necesidad de definirla.

NOTA: no se puede transformar el número entero a String o viceversa para resolver esta pregunta.



Nombre y Apellidos:
Sección:

RUT:
Firma:

Solución Pregunta 2:

(A) Asignación de puntajes:

- **(0.2p)** Contrato
- **(0.1p)** Propósito + ejemplo
- **(0.1p)** Encabezado función
- **(0.2p)** Testing
- Cuerpo función:
 - **(0.4p)** Obtener largo del número
 - **(1.0p)** Obtener extremo izquierdo
 - **(0.4p)** Obtener extremo derecho
 - **(0.6p)** Devolver suma

Implementación de ejemplo:

```
# sumarExtremos: int -> int
# entrega la suma de los pares de dígitos a los extremos del número
# ej: sumarExtremos(12453164) entrega 76
def sumarExtremos(N):
    largoNum = digitos(N)
    der = N % 100
    izq = N // (10 ** (largoNum - 2))
    return izq + der

# test
assert sumarExtremos(75412698) == 173
assert sumarExtremos(12453164) == 76
assert sumarExtremos(5642) == 98
assert sumarExtremos(657642461223) == 88
```



Nombre y Apellidos:
Sección:

RUT:
Firma:

(B) Asignación de puntajes:

- **(0.2p)** Contrato
- **(0.1p)** Propósito + ejemplo
- **(0.1p)** Encabezado función
- **(0.2p)** Testing
- Cuerpo función:
 - **(0.4p)** Obtener largo del número
 - Caso base:
 - **(0.1p)** Condición
 - **(0.1p)** Return
 - Caso recursivo:
 - Calcular nuevo N:
 - **(0.2p)** Eliminar extremo izquierdo
 - **(0.2p)** Eliminar extremo derecho
 - **(0.6p)** Comparar sumas de extremos
 - **(0.4p)** Devolver False si las sumas son distintas
 - **(0.4p)** Llamado recursivo a número reducido

Implementación de ejemplo:

```
# comprobar: int -> bool
# indica si el número cumple la 2ª propiedad de sumeru
# ej: comprobar(12453164) entrega True
def comprobar(N):
    largoNum = digitos(N)
    # Caso base
    if largoNum == 4:
        return True

    # Caso Recursivo: eliminar izq y luego der
    nuevoN = N % (10 ** (largoNum - 2))
    nuevoN = nuevoN // 100
    # alternativa: eliminar der y luego izq
    # nuevoN = N // 100
    # nuevoN = nuevoN % (10 ** (largoNum - 4))

    # Comparar la suma de los extremos, y ver como continuar
    if sumarExtremos(N) != sumarExtremos(nuevoN):
        return False
    else:
        return comprobar(nuevoN)

# test
assert not comprobar(75412698)
assert comprobar(12453164)
assert comprobar(5642)
assert comprobar(657642461223)
```